Experiment 9

Student Name: Chandan Kumar **UID: 23BAI70147**
Branch: BE CSE **Section/Group: 23AIT_KRG-G- 1**
Semester: 6th **Date of Performance: 17/04/26**
Subject Name: Full Stack - II **Subject Code: 23CSH - 382**

1. **Aim:**
   Secure & Scalable Full Stack System

2. **Objective:**

   By the end of this experiment, students will be able to:

   • Implement backend security using Spring Security and filter chains • Enable authentication using OAuth (Google Login)
   • Apply Role-Based Access Control (RBAC) for endpoint protection
   • Optimize database interactions using JPA and performance techniques • Integrate a secure backend with a React frontend using CORS

3. **Implementation/Code:**

   Product.java:

```java
package com.experiment9.entity;

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;

@Entity
@Table(name = "products", indexes = {
    @Index(name = "idx_product_name", columnList = "name"),
    @Index(name = "idx_product_category", columnList = "category")
})
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@NamedQueries({
    @NamedQuery(name = "Product.findByCategory",
                query = "SELECT p FROM Product p WHERE p.category = :category"),
    @NamedQuery(name = "Product.findByPriceRange",
                query = "SELECT p FROM Product p WHERE p.price BETWEEN :minPrice AND :maxPrice")
})
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(length = 500)
    private String description;

    @Column(nullable = false)
    private Double price;

    @Column(length = 50)
    private String category;

    @Column(nullable = false)
    private Integer stockQuantity;
}
```

Role.java:

```java
package com.experiment9.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "roles")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    @Column(length = 20, unique = true, nullable = false)
    private ERole name;
}
```

User.java:

```java
package com.experiment9.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.*;

import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "users",
        uniqueConstraints = {
            @UniqueConstraint(columnNames = "username"),
            @UniqueConstraint(columnNames = "email")
        })
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 50)
    private String username;

    @NotBlank
    @Size(max = 100)
    @Email
    private String email;

    @Size(max = 120)
    private String password;

    private String provider;   // "local" or "google"
    private String providerId; // Google sub-id

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
               joinColumns = @JoinColumn(name = "user_id"),
               inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles = new HashSet<>();
}
```

4. Learning Outcomes:

1. **Understand and implement backend security mechanisms** using Spring Security, including configuration of security filter chains and request authorization.
2. **Integrate OAuth-based authentication** (such as Google Login) to enable secure third-party user authentication in a full-stack application.
3. **Design and enforce Role-Based Access Control (RBAC)** to restrict access to APIs and resources based on user roles and permissions.

4. **Develop scalable backend services** using Spring Boot by applying best practices for modular and maintainable architecture.
5. **Optimize database operations using JPA**, including efficient querying, lazy/eager loading strategies, and performance tuning techniques.
6. **Configure secure communication between frontend and backend** by properly implementing CORS policies.
7. **Integrate a React frontend with a Spring Boot backend** while maintaining secure data exchange and authentication flow.
8. **Analyze and improve application performance and security**, identifying potential vulnerabilities and bottlenecks in full-stack systems.